

SangHyun Bang

방상현

Java · Spring Boot Backend Developer

github.com/sanghyunbang betterworldwithlucas.com

I find where data and state drift apart, block it with structure, and pin it down with tests. I read real SQL, query plans and logs to fix bottlenecks and structural flaws, and I check what a test actually verifies rather than trusting a green run. My business/economics background and data-analysis experience help me consider business and data flow together.

EXPERIENCE

레이시스 Backend Developer	2026.07 - present
ERP accounting module · Node.js, MariaDB	
Chung-Ang Univ. Industry-Academic Cooperation Foundation Student Researcher	2023.09 - 2024.08
Assisted Multi-Armed Bandit / reinforcement-learning research — theory analysis, experiment organization, project support	
Chung-Ang University Research Intern	2023.01 - 2023.08
Bandit Algorithm / Reinforcement Learning research intern	
Daol Investment & Securities Management Planning Intern	2022.01 - 2022.02
Supported management-planning work — experienced how business and data connect to decisions	

PROJECTS

Pawloop — dog-walking & family-care app (solo) 2026.06 - present Case study ↗ ONE store ↗ Source is private · demo on request
- Planning, Flutter app, Spring Boot server and AWS deployment · released on ONE store (2026.09) - Per-room chat sequence — reading the room and writing +1 lets two requests read the same value and hit the per-room unique constraint. Bundled the increment and "get my number" into UPDATE ... SET last_seq = LAST_INSERT_ID(last_seq + 1): the UPDATE's row lock serializes the requests, and the incremented value stays in the session, so the number comes back without re-reading the row - Pinned with a test on real MySQL (Testcontainers): 20 threads sending to the same room get sequence numbers 1–20 exactly once each (connection pool of 10, so at most 10 concurrent transactions). Resend idempotency uses a (room_id, sender_user_id, client_msg_id) unique constraint, with three cases fixed by tests: same ID duplicated / same key from another member / no key - First production deploy (2026.09.17): Caddy exited on a config parse error and ports 80/443 did not respond — the test guarding the config only checked Caddyfile strings and passed a broken config. Replaced it with a test that runs caddy validate in the production Caddy image via Testcontainers - Modular monolith on Spring Modulith — introduced boundary checks after the fact, cut violations from 620 to 107 and locked them with a ratchet test that fails if they grow; zero dependency cycles between modules - Full backend test run: 1,994 tests, 0 failures (as of 2026-09-30). About 81% of commits (445/550) were written with an AI coding agent and are marked with commit trailers - Java 21 / Spring Boot 3.5 / Spring Modulith / JPA / MySQL 8 / Testcontainers / Docker Compose · Caddy / AWS EC2 · Terraform / Flutter
SCOUT 2026.05 - present (solo · pre-launch) Case study ↗
A valuation service that estimates a fair-value range with public valuation methods (DCF · Monte Carlo), written with an AI coding agent (co-authorship marked in commits). Pure Kotlin engine + Spring Boot API (PostgreSQL · Testcontainers) + Flutter app. A private project being prepared as a business; the source is not public. Not investment advice.
BARO FARM — MSA commerce backend (member/auth, Gateway, OPA authorization) 2025.11.29 - 2026.01.27 (team project) Case study ↗
- Worked on the member/auth service, Gateway and OPA authorization in an MSA on Spring Cloud Gateway-Redis-Kafka-OPA - Diagnosed intermittent 403s as a structural "JWT permission vs user-state mismatch," not a gateway error, and designed and implemented a Kafka hotlist event → OPA bundle refresh path to cut the permission lag to within the polling interval (10–60s) - Wrote the member-withdrawal event to an outbox inside the withdrawal transaction, published by a scheduler (FAILED after 5 failures) (Later solo refactoring, 2026.02–03) Split the admin seller-approval path into commit-then-async propagation and recorded the decision in an ADR

HeatTrip — Emotion-based travel recommendation (Team Lead)

2025.07 - 2025.11 (competition · team HitHeat, 3 people) · 2026.03 - 2026.06 (solo refactoring) [Case study ↗](#)
[ONE store \(listing record · server no longer running\) ↗](#)

 Excellence Award, 2025 Tourism Data Utilization Competition (Korea Tourism Organization, 2025.11)

- Scoped the LLM role — instead of feeding tens of thousands of places to the LLM, it picks only the category (cat3); actual places are filtered by cat3 and ranked by a scoring function inside Spring
- Split place-list pagination into Offset / Cursor (createdtime + contentid composite key, Base64 cursor, size+1 for hasNext)
- (Solo refactoring) Place search — EXPLAIN ANALYZE showed the bottleneck was the count query (full scan + repeated correlated subquery, ~755ms); switched to search_text denormalization + FULLTEXT (ngram) + MATCH...AGAINST. Per the documented plan the count full scan is gone; list ~4.9ms (single measurement for '카페')
- (Solo refactoring) Added unit tests and observability code — AOP request/exception collection, fingerprint dedup → Slack alerts, correlation id. JaCoCo coverage reports · Qodana (GitHub Actions)
- Java 21 / Spring Boot 3.5 / Spring Security·OAuth2·JWT / JPA / MySQL 8 / AWS S3·CloudFront / Docker Compose

OREUM — map-based hiking community web service (team of 4)

2025.06 - 2025.07 (KDT graduation project) [Case study ↗](#)

 Excellence Award, KDT Full-stack Bootcamp graduation project (Gangnam HiMedia, 2025.07)

- Backend and frontend for the map (mountain search · mountain forecast · trails), curation posts, social login (OAuth2 · JWT) and S3 upload (Spring Boot · React)
- Forecasts for 153 mountains collected in batches and kept in Redis (12-hour TTL); the map's forecast API reads only from Redis (the automatic refresh trigger is off in the final code)
- Curation posts keep post metadata in MySQL and route segments (GeoJSON coordinates) in MongoDB, linked by postId and drawn as a route on the map

TRAINING

Programmers Dev Course · MSA Spring AI Intensive

Spring Cloud · Spring AI · MSA backend — BARO FARM project

2025.11 - 2026.01

Gangnam HiMedia · KDT Full-stack Bootcamp

Spring · React full-stack — Excellence Award for the graduation project (OREUM, hiking community)

2025.01 - 2025.07

EDUCATION

Academic Credit Bank System

B.Eng. in Computer Engineering · 4.05 / 4.5 · conferred 2026.08

2025.07 - 2026.08

Sogang University

Business Administration & Economics, double major · Summa Cum Laude · 3.75 / 4.3

2015.03 - 2023.02

Chung-Ang University Graduate School

M.S. program in Artificial Intelligence · on leave

2023.09 -

SKILLS

Backend — Java, Kotlin, Spring Boot, Spring Security, Spring Cloud Gateway, Spring Modulith, JPA / Hibernate, Node.js

DB / Messaging — MySQL, MariaDB, PostgreSQL, Redis, Kafka

Infra / Test — Docker Compose, Caddy, AWS (EC2 · S3 · CloudFront), Terraform, OPA, Testcontainers, GitHub Actions

Data / AI — Python, LLM integration, Data Analysis, Reinforcement Learning

Mobile / Etc. — Flutter, Git

CERTIFICATIONS

- Engineer Information Processing (2026.06)
- SQL Developer (SQLD) (2026.06)
- ADsP — Data Analysis Semi-Professional (2022.09)
- Certified Investment Manager (2022.12)

Language — IELTS 7.0 (2024.12) · OPIc IH (2026.03)

AWARDS

- Excellence Award, 2025 Tourism Data Utilization Competition (Korea Tourism Organization, 2025.11) — HeatTrip
- Excellence Award, KDT Full-stack Bootcamp graduation project (Gangnam HiMedia, 2025.07) — OREUM
- Honorable Mention, 13th DB FEC (DB Kim Jun-ki Cultural Foundation, 2023.05)
- Bank of Korea Monetary Policy Competition — preliminary round, top 11 teams in Seoul (2022.07)